

SEO PACKAGE

1. SEO Title

Maximize Active Section With Trade I — O(n) Python Solution (LeetCode Explained)

2. SEO Subtitle

Learn how a single pass over run-lengths beats brute-force simulation for LeetCode's "Maximize Active Section with Trade I."

3. Featured Quote

"Whatever you erase, you rebuild — the only real gain is the bread around the sandwich."

4. Meta Description

Solve LeetCode's Maximize Active Section with Trade I in O(n) using run-length encoding in Python. Full intuition, code, dry run, and edge cases.

5. URL Slug

maximize-active-section-trade-i-python-on

6. Keywords

LeetCode Python, Maximize Active Section with Trade I, binary string interview question, run-length encoding Python, groupby itertools, O(n) string algorithm, coding interview algorithms, prefix sum vs run length, brute force vs optimal, Daily Python LeetCode, string manipulation interview, data structures and algorithms, Google coding interview, Amazon coding interview, greedy algorithm string, segment scoring technique, binary string problems, Python interview prep, LeetCode string problems, algorithm complexity analysis

7. Tags

Python, LeetCode, Coding Interview, Algorithms, Data Structures, Software Engineering, String Manipulation, itertools, Interview Prep, DSA, Problem Solving, Big O Notation, Daily Python LeetCode

MAIN ARTICLE

Introduction

Binary string problems show up constantly in coding interviews because they look deceptively simple — a string of 0s and 1s — while quietly testing whether you can spot structure instead of reaching for brute force. "Maximize Active Section with Trade I" is a great example. On the surface it asks you to simulate a single swap operation. Underneath, it's really testing whether you understand run-length thinking: the idea that consecutive repeated characters can be collapsed into a handful of numbers, and that most "simulate every possibility" problems secretly reduce to a single arithmetic pass.

This pattern isn't just an interview trick. The same idea — grouping runs of identical values and reasoning about their boundaries — powers things like free-space coalescing in memory allocators, run-length encoding in image compression, and disk block management. Learning to recognize it here will help you recognize it again in a dozen other problems.

By the end of this article, you'll understand why the brute-force approach breaks down at scale, how to reframe the problem in terms of "runs" instead of individual characters, and how to write a clean $O(n)$ Python solution that never even needs to touch the actual string.

Problem Overview

You're given a binary string `s` made up only of `'0'` and `'1'` characters. Think of `'1'` as "active" and `'0'` as "inactive."

You may perform **exactly one operation**: choose a maximal group of consecutive `1`s that is directly bordered by a `0` on both sides (a group that is "boxed in"), erase it, and then fill the entire span — the erased `1`s and both bordering `0`s — with `1`s.

Your task is to return the **maximum number of active (`1`) positions** achievable after performing this operation at most once. If no valid group exists, the answer is simply the count of `1`s already in the string.

Example

Take `s = "1000100"`.

- Original active count: 2 (the two `1`s).

- There's a group of 1 s in the middle ("1000100" → index 4), boxed in by three 0 s on the left and two 0 s on the right.
- Trading it: erase that group, fill the whole span (both zero-runs plus the middle 1) with 1 s.
- Result: "1111111" → 7 active positions.

Second example: `s = "0100"` .

- Original active count: 1.
- The single 1 at index 1 is boxed in by one 0 on the left and two 0 s on the right.
- Trade it: the whole span becomes 1 s → "1111" → 4 active positions.

Every output makes sense once you realize the operation doesn't really "erase and refill" — it just merges a 1 -block with its two neighboring 0 -blocks into one long run of 1 s.

Intuition

Here's the mental shortcut that unlocks this problem: **the block of 1 s you erase gets turned right back into 1 s by the same operation.** It cancels itself out completely. Nothing about that middle block changes the final count.

So what actually changes? Only the 0 s immediately to the left and right of that block — those flip from inactive to active. That's the entire gain.

Picture it as a sandwich: the 1 -block is the filling, and the 0 -runs on either side are the bread. Trading turns the bread into filling too. Your gain from any single trade is just **(size of left zero-run) + (size of right zero-run)**.

Once you see that, the problem stops being about strings and starts being about **runs**. Instead of reading the string character by character, group it into runs of consecutive identical characters ("1000100" → 1 , 000 , 1 , 00). Now you just need the biggest sum of two adjacent zero-runs sandwiching a one-run, added to your starting count of 1 s.

To avoid special-casing the very first and last runs (what if a 1 -block sits right at the edge with no bordering 0 on one side?), pad the string with a '1' sentinel on each end before computing runs. That guarantees every real run has neighbors on both sides, and it can never accidentally qualify as a "boxed in" candidate itself since it's part of the padding.

Brute Force Solution

Idea: For every maximal 1-block that has 0 s on both sides, actually perform the swap — build the new string, count the 1 s, and track the best result.

Algorithm: 1. Scan the string to find every boxed-in 1-group. 2. For each candidate, construct the resulting string (or simulate the count directly). 3. Track the maximum active count seen.

Advantages: Simple to reason about, easy to verify by hand, useful as a first correct answer in an interview to show you understand the problem before optimizing.

Disadvantages: Recomputing or rescanning for every candidate group is wasteful — each candidate potentially costs $O(n)$ work, and there can be $O(n)$ candidates.

Complexity: $O(n^2)$ time in the worst case (many candidate groups, each requiring an $O(n)$ rescan), $O(n)$ space for the working string.

Optimal Solution

The optimal approach never rewrites the string at all — it only counts.

1. **Pad the string** with a '1' on each end. This guarantees the first and last runs are always padding, never real candidates, and removes all edge-case branching.
2. **Run-length encode** the padded string using `itertools.groupby`, producing pairs like ('1', 1), ('0', 3), ('1', 1), ('0', 2), ('1', 1).
3. **Record the baseline:** count of 1 s in the *original, unpadded* string.
4. **Walk the runs, skipping the first and last** (the sentinels). For every run of 1 s, its gain is (previous run length) + (next run length), since neighboring runs are guaranteed to be 0-runs.
5. **Track the maximum gain** across all interior 1-runs.
6. **Answer** = baseline 1-count + best gain (0 if no valid 1-run exists).

Run Type Length Gain if traded

pad	1	1	— (sentinel)
1	0	3	—
2	1	1	$3 + 2 = 5$
3	0	2	—
pad	1	1	— (sentinel)

For "1000100": baseline = 2, best gain = 5, answer = 7. ✓

Python Code

```

from itertools import groupby

def max_active_section(s: str) -> int:
    """Return the max active (1) count after at most one boxed-in trade."""
    baseline_ones = s.count("1")

    padded = "1" + s + "1"
    runs = [(char, len(list(group))) for char, group in groupby(padded)]

    best_gain = 0
    for i in range(1, len(runs) - 1): # skip sentinel runs at both ends
        char, length = runs[i]
        if char == "1":
            left_zeros = runs[i - 1][1]
            right_zeros = runs[i + 1][1]
            best_gain = max(best_gain, left_zeros + right_zeros)

    return baseline_ones + best_gain

```

Code Walkthrough

- `s.count("1")` grabs the starting active count from the **original** string, before padding — the padding is only a scanning convenience, it shouldn't count toward the answer.
- `padded = "1" + s + "1"` adds sentinel bumpers so every real `1` -run has a bordering run on each side, eliminating edge-case checks.
- `groupby(padded)` streams through the string and groups consecutive identical characters; wrapping each group in `list(group)` gives us its length.
- The loop bounds `range(1, len(runs) - 1)` explicitly skip the first and last runs, which are always the sentinel `1` s.
- Inside the loop, we only care about `1` -runs, since only trading a `1` -block produces a gain. Its neighbors are guaranteed to be `0` -runs (runs alternate by construction).
- `best_gain` tracks the single largest sandwich found; the string itself is never mutated.

Dry Run

```
s = "00100"
```

1. `baseline_ones = 1` .
2. `padded = "1001001"` — wait, let's be precise: `"1" + "00100" + "1" = "1001001"` .
3. Runs: `('1',1), ('0',2), ('1',1), ('0',2), ('1',1)` .
4. Skip index 0 and index 4 (sentinels).

5. Index 1: `('0', 2)` — not a 1-run, skip.
6. Index 2: `('1', 1)` — left neighbor `('0', 2) = 2`, right neighbor `('0', 2) = 2` → gain = 4.
`best_gain = 4`.
7. Index 3: `('0', 2)` — skip.
8. Answer = `1 + 4 = 5`.

Matches the expected result: `"00100"` → trade the middle 1 → `"11111"` → 5.

Complexity Analysis

- **Time:** $O(n)$. `s.count("1")` is $O(n)$, padding is $O(n)$, `groupby` is a single linear pass producing $O(n)$ total run elements, and the loop over runs is bounded by the number of runs, which is at most $O(n)$. All steps are linear and sequential, so total time is $O(n)$.
- **Space:** $O(n)$ in the worst case for the `runs` list (e.g., an alternating string like `"010101..."` produces $O(n)$ runs), though in practice it's $O(\text{number of runs})$, which is often much smaller than n .
- **Why correct:** Every possible trade's gain is fully determined by its two adjacent zero-run lengths, since the 1-block being traded contributes zero net change. Taking the max over all interior 1-runs finds the best possible trade in one pass, and the padding guarantees no valid candidate is missed or double-counted at the boundaries.

Alternative Solutions

Prefix sum approach: Precompute, for every index, how many consecutive 0s extend to the left and to the right. Then for each 1-block, look up the boundary values in $O(1)$. This achieves the same $O(n)$ time complexity but requires maintaining two additional $O(n)$ arrays instead of one compact runs list. It's a reasonable alternative, but for this problem the run-length approach is simpler to implement and easier to reason about — there's no strong reason to prefer prefix sums unless you're already building them for another part of a larger solution.

Edge Cases

- **All zeros** (`"0000"`): no 1-run exists at all, `best_gain` stays 0, answer is 0.
- **All ones** (`"1111"`): every run is 1, but a 1-run adjacent to another 1-run (not 0) never happens after `groupby`, and there are no 0-runs to sandwich against — `best_gain` stays 0, answer equals the original count.

- **Single character** ("0" or "1"): padding produces "101" or "111" ; no interior boxed-in group exists, answer is the original count.
- **Empty string**: `s.count("1")` is 0, padded string is "11" , runs are just the two sentinels, no interior runs to check, answer is 0.
- **Multiple candidate 1 -blocks of equal gain**: `max()` naturally picks any one of them since we only need the best value, not which block produced it.
- **1 -block touching the true edge of the original string** (e.g., "100"): after padding, this block is no longer adjacent to the sentinel directly — it still has a real 0 -run neighbor on one side and the sentinel on the other, correctly treated as boxed in.

Common Mistakes

1. **Forgetting to pad the string**, leading to index-out-of-range errors or incorrect special-casing for 1 -blocks that touch the actual start or end of `s` .
2. **Counting 1 s from the padded string instead of the original**, which inflates the baseline by 2.
3. **Failing to skip the sentinel runs** in the loop, which can incorrectly treat the padding itself as a candidate.
4. **Actually rewriting the string for each candidate** (brute-force habit) instead of just tracking run lengths, turning an $O(n)$ solution into $O(n^2)$.
5. **Assuming every 1 -run has two 0 -run neighbors without the padding trick**, which breaks for 1 -blocks at the true start or end of the string.
6. **Confusing "boxed in" with "any 1 -run"** — a 1 -run adjacent to another 1 -run can't happen after run-length encoding, but forgetting this alternation can lead to redundant checks.

Interview Questions

- How would you extend this to allow **k trades** instead of exactly one?
- What if the string could contain characters other than '0' and '1' ?
- Can you solve this without `itertools.groupby` , using manual index tracking?
- How would the prefix-sum variant change if the input string could be updated dynamically (point updates)?
- What's the maximum possible answer, and can you prove your solution never overcounts?

Similar Problems

- **LeetCode 485 — Max Consecutive Ones**: foundational run-counting on binary strings.

- **LeetCode 763 — Partition Labels:** another problem solved by tracking boundaries of grouped runs.
- **LeetCode 2760 — Longest Even Odd Subarray With Threshold:** practice reasoning about contiguous run boundaries.
- **LeetCode 1446 — Consecutive Characters:** direct run-length encoding practice.
- **LeetCode 838 — Push Dominoes:** another "segment and simulate boundaries" style problem.

Key Takeaways

Whenever a problem asks you to simulate an operation on a string of repeated values, ask whether the operation actually changes anything in the middle, or only at the boundaries. Collapsing a string into runs turns an $O(n^2)$ simulation into a single $O(n)$ pass. The padding trick — adding sentinel values at both ends — is a reusable habit that eliminates an entire category of edge-case bugs, not just in this problem but anywhere you're scanning for boundaries.

Watch the Video

For the full step-by-step walkthrough with visual tracing on "1000100", watch the video here: {LINK}

About the Series

This breakdown is part of the **Daily Python LeetCode** series, where each problem gets a complete treatment: brute force, optimized solution, complexity analysis, and the interview reasoning behind every step — all in clean, production-quality Python.

Call To Action

If this helped the idea click, subscribe on YouTube for the daily walkthroughs, subscribe to the Substack for the full written breakdowns, and drop a comment with which part clicked (or didn't) for you. Sharing this with someone prepping for interviews helps more than you'd think.

SOCIAL MEDIA

LinkedIn Post

Most engineers' first instinct on "Maximize Active Section with Trade I" is to simulate the swap on every candidate block. It works — until the string hits 100,000 characters and that $O(n^2)$ approach grinds to a halt.

The unlock is realizing the operation doesn't actually change what you erase. A block of 1s that gets erased is immediately rebuilt as 1s — it's a wash. The only real gain comes from the 0s bordering it on each side, which flip to active.

Once you see that, the whole problem collapses: pad the string with sentinel 1s to kill edge cases, run-length encode it, and take the max sum of two adjacent zero-runs around any 1-block. One pass, $O(n)$, done.

This pattern — grouping repeated values into runs and reasoning about their boundaries — isn't just an interview trick. It's the same idea behind free-space coalescing in memory allocators and run-length compression in image formats.

Full breakdown with code, dry run, and edge cases in the comments/link. Always happy to hear how others approached this one.

Python #LeetCode #CodingInterview

#Algorithms #DataStructures

Twitter/X Thread

1/ "1000100" has 2 active bits. One legal move later, it has 7. No magic — just one clean trick. Thread on LeetCode's "Maximize Active Section with Trade I" 

2/ The move: find a group of 1s boxed in by 0s on both sides, erase it, refill the whole span with 1s. Brute force tries this on every candidate. Works, but it's $O(n^2)$.

3/ Here's the insight that kills the brute force: whatever block of 1s you erase gets turned right back into 1s. It's a wash. Nothing changes there.

4/ So what actually changes? Only the 0s bordering that block. Those flip to 1s. That's the ENTIRE gain from any trade.

5/ Think sandwich: the 1-block is the filling, the bordering 0-runs are the bread. Trading turns bread into filling. Gain = size of the bread.

6/ Now reframe the whole string as "runs" instead of characters: "1000100" → runs of 1, 000, 1, 00. Suddenly you're doing arithmetic on a handful of numbers, not scanning repeatedly.

7/ One more trick: pad the string with a '1' sentinel on each end first. Now every real block has neighbors on both sides — zero edge-case branching needed.

8/ The algorithm: run-length encode the padded string, skip the two sentinel runs, and for every interior 1-run take (left zero-run + right zero-run). Track the max.

9/ Answer = original 1-count + best gain. Never rewrite the string — just count. That's $O(n)$ time, one pass, done.

10/ Full code + dry run + edge cases in the article/video. If you've hit this problem, what was your first instinct — brute force or straight to runs? {LINK}

Facebook Post

Ever notice how some coding problems look way scarier than they actually are? Today's one: you've got a string of 0s and 1s, and you're allowed one move — find a group of "on" switches boxed in by "off" switches, and flip the whole boxed-in area to "on."

Turns out you don't need to simulate anything to solve it fast. The trick is realizing the group you erase gets rebuilt anyway, so the only real gain is from the switches sitting right next to it. Group the string into "runs" (streaks of the same value), and the answer falls out in one pass.

Full walkthrough with Python code and a practice example in the link — good one if you're brushing up for interviews. {LINK}

Reddit Summary

Worked through LeetCode's "Maximize Active Section with Trade I" — sharing the approach since the brute force vs. optimal gap here is a nice teaching example.

The operation: erase a 1-block that's boxed in by 0s on both sides, then fill that whole span with 1s. Naive approach simulates this for every candidate block — $O(n^2)$, falls over at $n=1e5$.

Key realization: the block you erase gets rebuilt as 1s regardless, so it contributes zero net change. The only actual gain comes from the 0-runs immediately bordering it. So the problem reduces to: pad the string with 1 sentinels (kills edge cases), run-length encode it with `itertools.groupby`, and take the max of (left zero-run + right zero-run) over every interior 1-run. Add that to your baseline 1-count. $O(n)$ time, no string mutation.

There's also a prefix-sum variant (precompute left/right zero-run lengths per index) that's the same complexity class but needs two extra arrays instead of one runs list — runs felt like less code for the same result.

Happy to discuss alternative approaches or edge cases people ran into.

GITHUB README

```
# Maximize Active Section with Trade I – Python Solution

## Problem
Given a binary string `s`, you may perform exactly one operation: erase a maximal group of `1`s that is bordered by `0`s on both sides, then fill the entire span (the erased `1`s and both bordering `0`s) with `1`s. Return the maximum number of active (`1`) positions achievable.

## Intuition
The block of `1`s you erase gets rebuilt as `1`s regardless – it contributes zero net change. The only real gain comes from the `0`-runs bordering it, which flip to active. So the problem reduces to finding the `1`-block whose two adjacent zero-runs sum to the largest value.

## Approach
1. Pad `s` with a `1` sentinel on each end to remove edge-case branching.
2. Run-length encode the padded string with `itertools.groupby`.
3. Skip the two sentinel runs; for every interior `1`-run, compute `left_zero_run + right_zero_run`.
4. Track the maximum gain, add it to the original `1`-count.

## Python Solution
```python
from itertools import groupby

def max_active_section(s: str) -> int:
 baseline_ones = s.count("1")

 padded = "1" + s + "1"
 runs = [(char, len(list(group))) for char, group in groupby(padded)]

 best_gain = 0
 for i in range(1, len(runs) - 1):
 char, length = runs[i]
 if char == "1":
 left_zeros = runs[i - 1][1]
 right_zeros = runs[i + 1][1]
 best_gain = max(best_gain, left_zeros + right_zeros)

 return baseline_ones + best_gain
```

\\\`

## ## Complexity

- **Time:**  $O(n)$  — one pass to count, one pass to run-length encode, one pass over the runs.
- **Space:**  $O(n)$  worst case for the runs list (bounded by number of runs).

## ## Video

Full walkthrough with dry run and edge cases: {LINK}

## ## Article

Complete written breakdown: see the linked Substack article for intuition, brute force comparison, dry run, and interview follow-ups.

## Quick Review

### What pattern fits 'flip one segment of 0s to maximize consecutive 1s'?

Segment/run-length grouping + sliding-merge: split into runs, then test merging across each 0-run using its neighbors.

### Time complexity for scanning runs and evaluating each possible trade?

$O(n)$ : one pass to build runs, one pass over runs to score each candidate merge.

### Space complexity for the run-length segment approach?

$O(n)$  worst case to store run lengths/positions, or  $O(1)$  if scanning with pointers instead of storing all segments.

### Trickiest edge case in this one-trade flip problem?

A 0-run at the string's start or end has only one neighboring 1-run (or none) — don't assume both sides exist.

### Brute-force every trade vs segment-scoring approach — why avoid brute force?

Trying every possible flip position is  $O(n^2)$ ; segmenting first turns it into  $O(n)$  by only checking boundaries between runs.

### Interview follow-up: why does erasing a 0 only ever merge two adjacent 1-runs?

Because a single 0 is the sole gap between them; removing it joins them into one contiguous run, no other 0s are affected.

Maximize Active Section with Trade I — free Python cheat sheet